

UNIVERSIDAD ANTONIO JOSE DE SUCRE
Programa de Ingeniería de Sistemas

ENTREGA 1
Documento de Requerimientos y Arquitectura Propuesta

SISTEMA DE GESTIÓN DE TUTORÍAS UNIVERSITARIAS

Curso:	Electiva profesional II
Docente:	Ing. Jaider Javier de la Rosa Bertel
Integrantes:	Wadid Rivas Payares
	Azael Espitia Toribio
	Jhon Jader Bertel Ortega
	Eduardo Julio Bettin Avilez
Fecha de entrega:	12/09/2026

1. Introducción y Planteamiento del Problema

En el contexto universitario, los estudiantes pueden requerir apoyo académico adicional para comprender determinados temas, fortalecer sus conocimientos o mejorar su desempeño en diferentes asignaturas. Las tutorías representan una alternativa importante para brindar este acompañamiento; sin embargo, su gestión puede realizarse mediante procesos manuales, mensajes informales, hojas de cálculo u otras herramientas que no están diseñadas específicamente para organizar la información relacionada con estudiantes, tutores, materias, horarios y solicitudes.

Esta situación puede afectar principalmente a los estudiantes, quienes pueden tener dificultades para conocer qué tutores se encuentran disponibles, qué materias ofrecen tutorías y en qué horarios pueden solicitar una sesión. De igual manera, los tutores pueden presentar dificultades para organizar las solicitudes recibidas, controlar sus horarios y realizar seguimiento al estado de cada tutoría. Por su parte, los administradores pueden encontrar inconvenientes para mantener actualizada la información de usuarios, materias y tutorías.

La ausencia de un sistema centralizado puede ocasionar problemas como pérdida de información, duplicidad de registros, dificultades para consultar la disponibilidad de los tutores y falta de seguimiento sobre las solicitudes realizadas. Además, el uso de diferentes medios para gestionar la información puede hacer que el proceso sea menos organizado y eficiente, afectando la comunicación entre estudiantes, tutores y responsables de la gestión académica.

Por esta razón, surge la necesidad de desarrollar un **Sistema de Gestión de Tutorías Universitarias**, que permita centralizar y organizar la información relacionada con el proceso de tutorías mediante una plataforma web. El sistema permitirá a los estudiantes consultar materias y tutores disponibles, realizar solicitudes de tutoría y consultar el estado de sus solicitudes; a los tutores les permitirá gestionar las solicitudes recibidas; y a los administradores les permitirá administrar la información principal del sistema.

La implementación de esta solución permitirá mejorar la organización del proceso de tutorías, facilitar el acceso a la información y proporcionar un mecanismo centralizado para gestionar las solicitudes. De esta manera, se busca reducir los inconvenientes derivados de los procesos manuales y ofrecer una herramienta tecnológica que contribuya a una gestión más eficiente y ordenada de las tutorías universitarias.

2. Objetivos

2.1 Objetivo general

Desarrollar un sistema web para la gestión de tutorías universitarias que permita a los estudiantes solicitar tutorías, a los tutores gestionar sus solicitudes y al administrador controlar la información principal del sistema, mediante una arquitectura cliente-servidor basada en Angular, Spring Boot y MySQL gestionado mediante AWS RDS.

2.2 Objetivos específicos

- **Diseñar** una interfaz web sencilla e intuitiva que permita a estudiantes, tutores y administradores interactuar con el sistema de acuerdo con sus funciones.
- **Implementar** un mecanismo de autenticación y control de acceso basado en roles, permitiendo gestionar usuarios, materias y tutores según los permisos establecidos.
- **Desarrollar** las funcionalidades necesarias para que los estudiantes puedan consultar materias y tutores disponibles, realizar solicitudes de tutoría y consultar el estado de sus solicitudes.
- **Implementar** la gestión de tutorías mediante una API REST desarrollada con Spring Boot, permitiendo a los tutores consultar, aceptar o rechazar solicitudes de tutoría y administrar la información relacionada con sus sesiones.
- **Configurar** la persistencia de la información utilizando MySQL y AWS RDS, y realizar pruebas funcionales de los principales módulos antes de la entrega.

3. Alcance y Limitaciones

3.1 Alcance (incluye)

- Autenticación y gestión de usuarios, incluyendo registro, inicio de sesión y control de acceso según los roles de estudiante, tutor y administrador.
- Gestión administrativa, permitiendo al administrador crear, consultar, editar, desactivar y gestionar usuarios, materias y tutores.
- Gestión de materias y tutores, permitiendo registrar y mantener actualizada la información necesaria para ofrecer las tutorías.
- Consulta de tutorías, permitiendo a los estudiantes consultar las materias, tutores y horarios de tutoría disponibles.

- Gestión de solicitudes de tutoría, permitiendo a los estudiantes crear, consultar y cancelar sus solicitudes, y a los tutores aceptarlas o rechazarlas.
- Consulta y seguimiento de estados, permitiendo visualizar el estado de las solicitudes y tutorías dentro del sistema.
- Arquitectura cliente-servidor, mediante una interfaz desarrollada en Angular y una API REST desarrollada con Spring Boot.
- Persistencia de información, utilizando MySQL y Spring Data JPA, con la base de datos desplegada en AWS RDS.
- Pruebas funcionales básicas de los principales módulos del sistema antes de la entrega del proyecto.

3.2 Limitaciones (no incluye)

- Chat o comunicación en tiempo real entre estudiantes y tutores.
- Funciones de inteligencia artificial, como asistentes virtuales, recomendaciones automáticas o generación de contenido académico.
- Notificaciones avanzadas mediante correo electrónico, SMS o servicios externos.
- Aplicación móvil nativa para Android o iOS; el proyecto se desarrollará como una aplicación web.
- Integraciones con otros sistemas universitarios externos, como sistemas académicos, plataformas de notas o servicios institucionales.

4. Requerimientos Funcionales

ID	Historia de usuario (Como... quiero... para...)	Prioridad	Módulo
RF-01	Como estudiante , quiero registrarme en el sistema, para crear una cuenta y acceder a las funcionalidades disponibles.	Alta	Autenticación y usuarios
RF-02	Como usuario , quiero iniciar sesión, para acceder a las funcionalidades correspondientes a mi rol dentro del sistema.	Alta	Autenticación y usuarios
RF-03	Como administrador , quiero gestionar usuarios y sus roles, para mantener actualizada y controlada la información de los usuarios del sistema.	Alta	Administración
RF-04	Como administrador , quiero gestionar materias y tutores, para mantener actualizada la oferta de tutorías disponibles.	Alta	Administración
RF-05	Como estudiante , quiero consultar las materias, tutores y tutorías disponibles, para seleccionar una opción adecuada a mi necesidad académica.	Alta	Consulta de tutorías

ID	Historia de usuario (Como... quiero... para...)	Prioridad	Módulo
RF-06	Como estudiante , quiero solicitar una tutoría disponible, para recibir apoyo académico en una materia determinada.	Alta	Solicitudes
RF-07	Como estudiante , quiero consultar y cancelar mis solicitudes de tutoría, para conocer su estado y evitar solicitudes que ya no necesite.	Media	Solicitudes
RF-08	Como tutor , quiero consultar las solicitudes de tutoría recibidas, para conocer y organizar las tutorías solicitadas por los estudiantes.	Alta	Gestión de solicitudes
RF-09	Como tutor , quiero aceptar o rechazar solicitudes de tutoría, para gestionar la disponibilidad de mis tutorías.	Alta	Gestión de solicitudes
RF-10	Como administrador , quiero consultar las tutorías y solicitudes registradas, para supervisar el funcionamiento general del servicio.	Media	Administración y supervisión

5. Requerimientos No Funcionales

 Instrucción: Complete al menos una fila por categoría. Los criterios deben ser medibles (ej.: 'la respuesta del API no debe superar 2 segundos').

ID	Categoría	Descripción / criterio medible
RNF-01	Seguridad	El sistema deberá almacenar las contraseñas utilizando un mecanismo de cifrado seguro y deberá restringir el acceso a las funcionalidades según el rol del usuario. En las pruebas, un usuario no podrá acceder a endpoints que no correspondan a su rol.
RNF-02	Rendimiento	Las operaciones principales de la API REST, como consultas de materias, tutores y solicitudes, deberán responder en un tiempo máximo de 3 segundos bajo una carga académica normal.
RNF-03	Usabilidad	La interfaz deberá permitir que los usuarios completen las operaciones principales de su rol sin asistencia externa, utilizando formularios claros, navegación consistente y mensajes de validación comprensibles.
RNF-04	Escalabilidad	La arquitectura deberá permitir incorporar nuevos módulos o funcionalidades mediante la estructura por capas de Angular y Spring Boot, sin requerir una modificación completa de los módulos existentes.
RNF-05	Disponibilidad	La base de datos deberá estar desplegada en AWS RDS y deberá poder ser accedida por la API durante las pruebas del sistema, permitiendo consultar y almacenar información de manera continua mientras el servicio se encuentre disponible.
RNF-06	Mantenibilidad	El backend deberá estar organizado en las capas Controller , Service , Repository y Entity , de manera que cada componente tenga una

ID	Categoría	Descripción / criterio medible
		<i>responsabilidad definida y facilite la modificación y mantenimiento del código.</i>
RNF-07	Validación de datos	<i>La API deberá validar los datos recibidos antes de procesarlos y deberá devolver respuestas de error apropiadas ante datos inválidos, evitando registrar información incompleta o incorrecta en la base de datos.</i>

6. Actores del Sistema

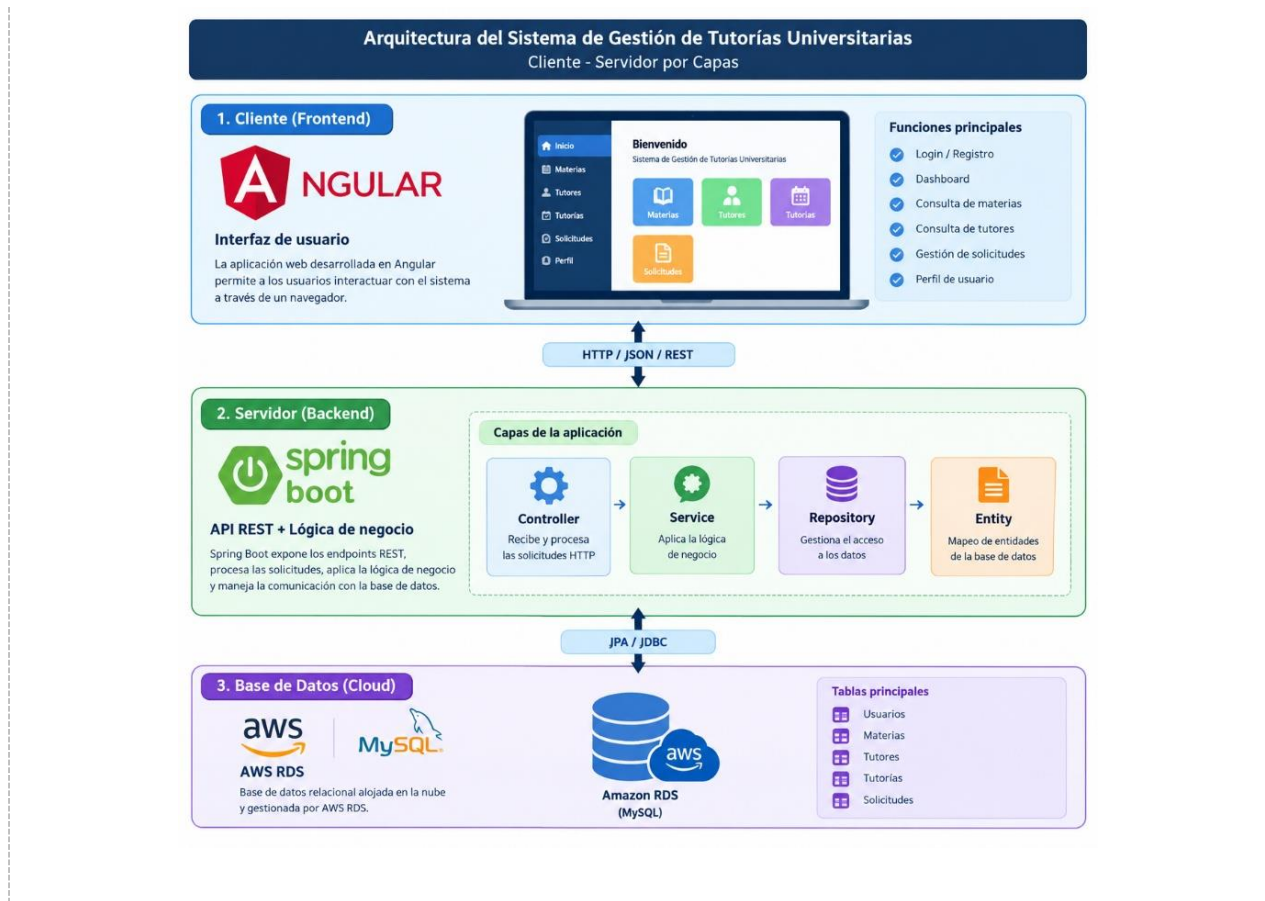
 Instrucción: Identifique todos los tipos de usuario del sistema y sus permisos diferenciados.

Actor / Rol	Descripción	Permisos principales
Administrador	<i>Es el usuario encargado de la administración y supervisión general del sistema. Tiene el mayor nivel de acceso y controla la información principal de la plataforma.</i>	<i>Gestionar usuarios, materias y tutores; consultar tutorías y supervisar la información principal del sistema.</i>
Tutor	<i>Es el usuario encargado de ofrecer tutorías y gestionar las solicitudes realizadas por los estudiantes de acuerdo con su disponibilidad.</i>	<i>Iniciar sesión, consultar solicitudes recibidas, aceptar o rechazar solicitudes y consultar sus tutorías.</i>
Estudiante	<i>Es el usuario que utiliza la plataforma para buscar apoyo académico y solicitar tutorías con los tutores disponibles.</i>	<i>Registrarse, iniciar sesión, consultar materias, consultar tutores, solicitar tutorías, consultar sus solicitudes y cancelar solicitudes.</i>

7. Arquitectura Propuesta

7.1 Diagrama de arquitectura

 Instrucción: Inserte aquí su diagrama de arquitectura (cliente-servidor por capas). Debe mostrar claramente: Angular (Frontend) ↔ API REST (Spring Boot) ↔ Base de datos MySQL en AWS RDS.



7.2 Descripción de las capas

- **Capa de presentación (Frontend – Angular):** Esta capa corresponde a la interfaz web con la que interactúan los usuarios del sistema. Será desarrollada utilizando **Angular**, HTML, CSS y TypeScript. Su función principal será presentar la información y permitir que cada usuario realice las operaciones correspondientes a su rol.
- Desde esta capa se podrán realizar funciones como el inicio de sesión, consulta de materias y tutores, gestión de solicitudes y visualización de tutorías. Angular se comunicará con el backend mediante solicitudes HTTP utilizando una **API REST**, intercambiando información en formato JSON.
- **Capa de lógica de negocio (Backend – Spring Boot):** Esta capa será desarrollada con **Spring Boot** y tendrá la responsabilidad de procesar las solicitudes provenientes del frontend, aplicar las reglas de negocio y controlar el acceso a las diferentes funcionalidades según el rol del usuario.
- El backend estará organizado mediante las capas **Controller**, **Service**, **Repository** y **Entity**. Los Controllers recibirán las solicitudes HTTP, los Services aplicarán la lógica de negocio, los Repositories gestionarán el acceso a los datos y las Entities representarán las tablas de la base de datos.

- Spring Boot también será responsable de exponer los diferentes endpoints de la **API REST**, validar la información recibida y comunicarse con la base de datos mediante Spring Data JPA.
- **Capa de datos (MySQL en AWS RDS):** Esta capa estará compuesta por una base de datos **MySQL alojada en Amazon RDS (AWS)**. Su función será almacenar de manera persistente la información del sistema, incluyendo usuarios, materias, tutores, tutorías y solicitudes.
- El acceso a la base de datos será realizado por el backend mediante **Spring Data JPA**, evitando que el frontend tenga acceso directo a la base de datos. AWS RDS permitirá disponer de la base de datos como un servicio administrado en la nube.

7.3 Justificación de la arquitectura

Se seleccionó una arquitectura cliente-servidor de tres capas porque permite separar claramente la interfaz de usuario, la lógica de negocio y la gestión de los datos. Esta separación facilita el desarrollo, las pruebas y el mantenimiento del sistema, además de permitir que cada componente tenga una responsabilidad específica.

El uso de Angular en la capa de presentación permite construir una interfaz web organizada e interactiva para los diferentes roles del sistema. La comunicación mediante una API REST facilita la separación entre el frontend y el backend y permite que ambos componentes evolucionen de manera independiente.

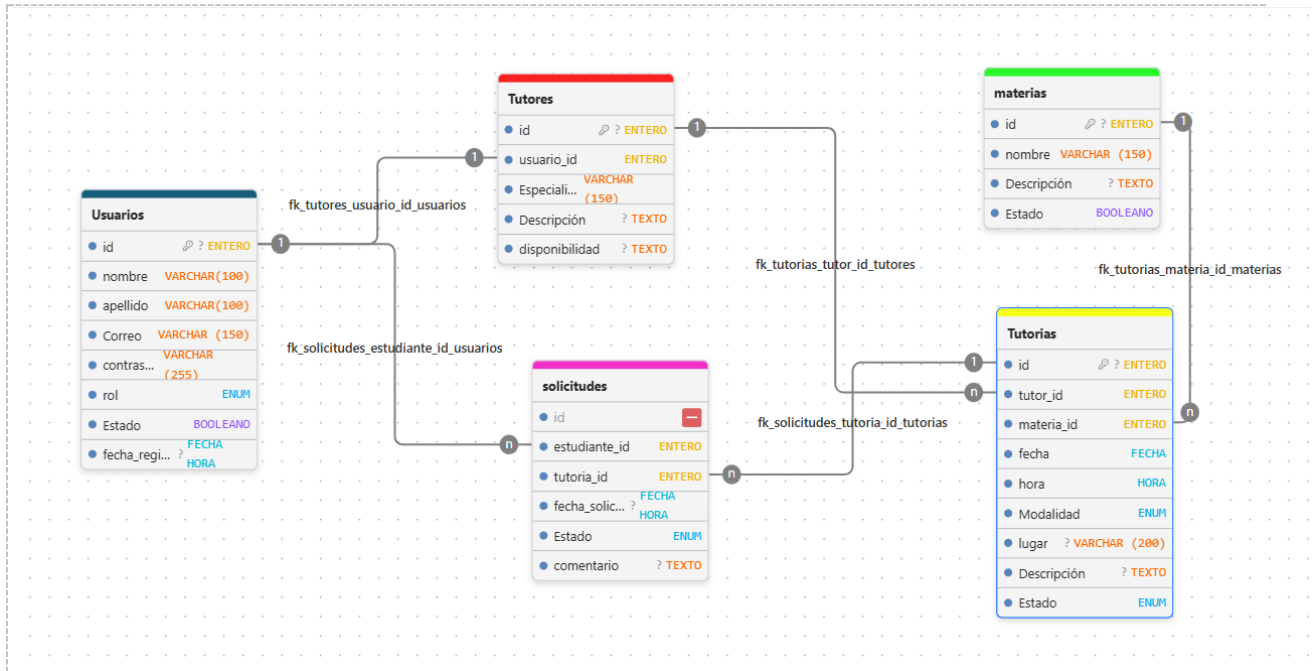
Spring Boot se utilizará como backend debido a que proporciona herramientas para desarrollar APIs REST, implementar la lógica de negocio, validar información y organizar el código mediante una arquitectura por capas. Esto favorece principalmente los requerimientos de seguridad, mantenibilidad y escalabilidad definidos para el proyecto.

Por otra parte, MySQL en AWS RDS permite almacenar la información de manera estructurada y persistente. Al utilizar Spring Data JPA se facilita el acceso y manejo de los datos desde el backend. El uso de AWS RDS también contribuye al requerimiento de disponibilidad, al utilizar un servicio de base de datos administrado en la nube.

En conjunto, esta arquitectura favorece el cumplimiento de los requerimientos no funcionales establecidos: permite implementar controles de seguridad y acceso por roles, mantener tiempos de respuesta adecuados mediante una separación de responsabilidades, facilitar la mantenibilidad del código, permitir la incorporación de nuevos módulos y utilizar una infraestructura de base de datos en la nube que favorece la disponibilidad del sistema.

8. Modelo Entidad-Relación Preliminar

 Instrucción: Administrador crea materia → Tutor crea tutoría → Estudiante consulta tutorías → Estudiante solicita → Tutor acepta/rechaza → Tutoría cambia de estado.



Entidad	Atributos principales	Relaciones
Usuarios	id, nombre, apellido, correo, password, rol, estado, fecha_registro	Se relaciona con Tutores mediante una relación 1:0..1 , ya que un usuario puede tener un perfil de tutor. Se relaciona con Solicitudes mediante una relación 1 , ya que un estudiante puede realizar varias solicitudes.
Materias	id, nombre, descripcion, estado	Se relaciona con Tutorías mediante una relación 1 , ya que una materia puede tener varias tutorías programadas.
Tutores	id, usuario_id, especialidad, descripcion, disponibilidad	Se relaciona con Usuarios mediante una relación 1:1 para el perfil de tutor y con Tutorías mediante una relación 1 , ya que un tutor puede ofrecer varias tutorías.
Tutorías	id, tutor_id, materia_id, fecha, hora, modalidad, lugar, descripcion, estado	Se relaciona con Tutores mediante una relación N:1 , con Materias mediante una relación N:1 y con Solicitudes mediante una relación 1 , ya que una tutoría puede recibir varias solicitudes.

Entidad	Atributos principales	Relaciones
<i>Solicitudes</i>	<i>id, estudiante_id, tutoria_id, fecha_solicitud, estado, comentario</i>	<i>Se relaciona con Usuarios (estudiantes) mediante una relación N:1 y con Tutorías mediante una relación N:1. Cada solicitud pertenece a un estudiante y a una tutoría.</i>

9. Stack Tecnológico y Justificación

Componente	Tecnología	Justificación
Frontend	Angular	<i>Se utilizará para desarrollar la interfaz web del sistema, permitiendo crear componentes reutilizables, organizar las diferentes vistas y gestionar la interacción de los estudiantes, tutores y administradores con la plataforma.</i>
Backend	Spring Boot	<i>Se utilizará para desarrollar la API REST y la lógica de negocio del sistema. Facilita la creación de servicios web, la organización del backend por capas y la comunicación con la base de datos mediante Spring Data JPA.</i>
Base de datos	MySQL (AWS RDS)	<i>MySQL permitirá almacenar de forma estructurada y persistente la información de usuarios, materias, tutores, tutorías y solicitudes. AWS RDS permitirá alojar la base de datos en la nube mediante un servicio administrado.</i>
Pruebas de API	Postman	<i>Se utilizará para probar los endpoints de la API REST desarrollada con Spring Boot, verificando operaciones como consultas, registros, actualizaciones, autenticación y manejo de solicitudes antes de integrarlas con Angular.</i>
Control de versiones	[Git / GitHub]	<i>Se utilizarán para controlar las diferentes versiones del código fuente, registrar los cambios realizados por los integrantes del equipo y facilitar el trabajo colaborativo durante el desarrollo del proyecto.</i>

10. Cronograma Interno del Equipo

 Instrucción: Distribuya roles y responsabilidades entre los integrantes. Puede ajustar los roles sugeridos según el tamaño real del equipo.

Integrante	Rol	Responsabilidades principales	Disponibilidad
Eduardo Julio	Líder técnico	Coordinar el desarrollo del proyecto, distribuir tareas, realizar seguimiento al cronograma, apoyar la integración entre frontend y backend y verificar que se cumplan los requerimientos establecidos.	[Horas/semana]
Azael Espitia	Encargado BD / AWS	Diseñar y configurar la base de datos MySQL, crear las tablas y relaciones, realizar la conexión con AWS RDS y verificar la persistencia de la información..	[Horas/semana]
Wadid Rivas	Encargado Backend	Desarrollar la API REST con Spring Boot, implementar la lógica de negocio, crear entidades, repositorios, servicios y controladores, y realizar pruebas de los endpoints.	[Horas/semana]
Eduardo Julio Bettin Avilez	Encargado Frontend	Desarrollar las interfaces principales en Angular, implementar componentes, formularios, navegación y vistas correspondientes a las funcionalidades del sistema.	[Horas/semana]
Azael Espitia Toribio	Encargado Frontend	Apoyar el desarrollo de las interfaces en Angular, implementar vistas según los roles del sistema, mejorar la usabilidad y realizar la integración de las pantallas con la API REST.	
Jhon Jader Bertel Ortega	Encargado QA / Documentación	Realizar pruebas funcionales del sistema, identificar y reportar errores, verificar el cumplimiento de los requerimientos y apoyar la elaboración y organización de la documentación del proyecto.	[Horas/semana]

11. Conclusiones Preliminares

 *Instrucción: Cierre el documento con 2-3 párrafos que resuman los principales hallazgos del análisis y los retos técnicos que anticipan para las siguientes entregas.*

El Sistema de Gestión de Tutorías Universitarias constituye una propuesta viable para centralizar y organizar el proceso de solicitud y administración de tutorías. El proyecto presenta un alcance controlado que permite desarrollar funcionalidades CRUD y aplicar una arquitectura cliente-servidor con tecnologías solicitadas en la actividad.

La arquitectura Angular ↔ Spring Boot ↔ MySQL en AWS RDS permite separar la presentación, la lógica de negocio y la persistencia de datos. Esta separación facilita el mantenimiento y permite que el equipo trabaje de manera distribuida durante el desarrollo.

Como siguiente paso, se deberá convertir este borrador en diseños UML definitivos, definir con mayor precisión las entidades y endpoints, crear el proyecto base y comenzar la implementación progresiva siguiendo el cronograma de diez días.